

ADITYA VANDAN SHARMA

@ adityavandan@outlook.com ☎ +91-7566665551 📍 Bangalore, India 🔗 adityavandan.github.io/
in linkedin.com/in/aditya-vandan-sharma/ 📄 github.com/AdityaVandan

EXPERIENCE

Software Engineer 2

Tesco

📅 March 2025 – Present 📍 Bangalore, India

- Built an AI triage agent that correlates Zendesk tickets with Splunk logs and live production APIs to identify root cause before an engineer opens the ticket, cutting first-response investigation from 45 mins to 5 mins on the 63% of the 500 tickets/week.
- Designed the retrieval layer around ripgrep and git pickaxe over shallow clones pinned to deployed SHAs, grounding every diagnosis in the code actually running in production rather than HEAD.
- Redesigned single-product induction (45 min for 3 products) into Bulk Induction, cutting onboarding time by 85% (15–20 min for 30+ products) across a 300K+ SKU catalog serving 15,000 supplier/buyer/persona users.
- Architected Bulk Induction: batched attribute fetching (up to 500 attributes/product), lazy-loaded rendering for large hierarchical payloads (3–5MB/product), and IndexedDB caching, while designing the workflow to support 50 products.
- Rolled out an AI agent across an 8-engineer team that auto-fixes failing tests from dev-session changes and opens PRs for review, saving 8–16 engineer-hours weekly
- Led a team of 2 engineers and mentored them

Software Engineer 2

Rippling

📅 July 2022 – Feb 2025 📍 Bangalore, India

- Architected and rolled out a Design Tokens system that became the org-wide standard for the design-to-development pipeline, adopted across both mobile and web teams.
- Optimized People Directory and Settings in mobile to improve performance by more than 50%
- Led the CI/CD overhaul for the mobile repo for screenshot testing, dead-code elimination, linting, testing, cross-repo dependency handling for cutting build friction and improving developer experience across the team.
- Architected and shipped an end-to-end Tax & Exemptions Correction Framework, automating the quarterly amendment process and eliminating 9–10 person-weeks of manual analyst and support-engineering effort every quarter.

Software Development Engineer

Groww

📅 October 2020 – July 2022 📍 Bangalore, India

- Built the Fixed Deposit product end-to-end across mobile and web driving 1 lakh FDs booked through the platform and critical mutual funds journeys with full technical ownership
- Optimised core website's chunking/tree shaking to reduce size of core chunk of website by 15%
- Worked on shipping first version of OSS Design System of Groww website

Data Engineer

Almug.ai

📅 June 2020 – Sept 2020 📍 Bangalore, India

SKILLS

Programming Languages : JavaScript, TypeScript, Python, Go, C++, Java

Frameworks & Libs : Django, FastAPI, ReactJS, NextJS, React-Native

Databases : PostgreSQL, MongoDB, Redis

AI/ML : LangChain, RAG, agent evaluation, LLM tooling

CI/CD : Git, Docker, Kubernetes, GitHub Actions, Jenkins

EDUCATION

Bachelor of Technology

Ujjain Engineering College, Ujjain

📅 June 2016 – June 2020

OPEN SOURCE

- Contributed to **unimported**, a cli tool to find dead dependencies and code.
Link: <https://github.com/smeijer/unimported>
- Contributed to **webster**, a mono repo consisting of Groww's Design System, Utilities and Other Charting Solutions.
Link: <https://github.com/Growww/webster>

PROJECTS

Atlas (Stock Trading Framework)

- Atlas is a Stock trading framework made for Trading, Forward Testing and Backtesting strategies. The project comprises of 2 separate segments: **Atlas-Go** and **Atlas-Quant**
- **Atlas-Go:** It is a framework for forward testing and actually making trades in real time. The framework is so created that it works with multiple brokers with very little config required for onboarding a new broker and also has a notification and logging system for the trades made.
- **Atlas-Quant:** It is a backtesting framework for testing strategies on existing data.
Resources used: Go, Zerodha Go SDK, Fyers APIs, PostgreSQL, Python, SQLAlchemy, Quant

Path-Finder Visualizer

- This project visualizes the different graph searching algorithms used in solving and creating a maze. The algorithms include: BFS, DFS, Dijkstra, Greedy BFS, A* and Swarm, along with some maze generating algorithms
- **Project Link:**
<https://adityavandan.github.io/path-finder>
Github Link: github.com/AdityaVandan/path-finder
Resources used: JavaScript ES6, React, Git

- Designed and Implemented a data processing pipeline for financial data from press releases of fortune 500 companies.